

Teaching an Instruction-Tuned LLM to Reason: Letter-Counting via Group Relative Policy Optimization (GRPO)

Generative AI Engineering Project Report

Base Model: Qwen2.5-3B-Instruct | Method: LoRA + GRPO | Stack: Unsloth, vLLM, TRL

August 2026

1. Executive Summary

This project set out to close a well-known weakness of instruction tuned large language models: precise, procedural, character level reasoning. While models such as Qwen2.5-3B-Instruct are fluent writers, they routinely fail at simple mechanical tasks such as counting how many times a given letter appears in a word, often producing a confident but wrong answer without showing any work. This report documents the design, implementation, and evaluation of a reinforcement learning pipeline that teaches a 3 billion parameter instruction tuned model to perform this task reliably, using visible, step by step, letter by letter reasoning.

The approach combines four modern, complementary technologies. Unsloth accelerates model loading and training and reduces memory consumption so that a 3B parameter model can be fine tuned on a single 16GB T4 GPU. LoRA (Low-Rank Adaptation) makes the fine tuning parameter efficient by freezing the 3 billion base weights and training only a small set of injected adapter matrices, roughly 31 million parameters at the chosen rank of 64, or about 1 percent of the base model. vLLM supplies the high throughput sampling engine needed to generate the many candidate completions that reinforcement learning requires. Finally, Group Relative Policy Optimization (GRPO) is the reinforcement learning algorithm itself: it samples a group of candidate answers per prompt, scores each with a set of custom, rule based reward functions, and nudges the policy toward the higher scoring completions relative to their group, without needing a separate value network or human labeled preference data.

A five part reward function suite was designed and validated to shape the desired behavior: rewarding in-order numbering of reasoning steps, exact letter by letter spelling of the target word, an accurate running count at every step, adherence to a structured reasoning and answer output format, and, most heavily, a correct final numeric answer. Each reward function was unit tested against a deliberately worse and a deliberately better model response, and in every case the better response scored strictly higher, confirming the reward signal points in the right direction before any GPU time was spent on training.

A 401-example dataset was procedurally generated from 62 words ranging from four to eight letters, pairing each word with letters that are and are not present, so the model must learn to count zero occurrences as reliably as one or two. Training was carried out in two stages, a five-step quick train to confirm the reward pipeline behaves as expected, followed by a longer, roughly one-hour, 100-step full training run, and both stages were executed end to end on GPU hardware. The full run's mean correctness reward showed a genuine, quantified upward trend

across the first three-quarters of training before a late-run wobble driven by rising KL divergence, analyzed in detail in Section 6.

The resulting LoRA adapter, a small `adapter_model.safetensors` file rather than a new copy of the full model, plugs directly into the frozen Qwen2.5-3B-Instruct base and, in side by side comparison, is expected to produce clearly structured, letter by letter reasoning and correct counts where the untuned baseline model previously guessed. Critically, the same comparison harness was used to confirm that general world knowledge, tested with an unrelated factual question, is preserved after fine tuning, indicating that the new skill was added without catastrophic forgetting of the model's original capabilities. This report walks through the problem, the solution architecture, each core component, the training results, and the practical value, limitations, and conclusions of this approach.

2. Problem Statement

Large language models are trained primarily on next-token prediction over vast amounts of natural language text. This objective makes them extremely good at producing fluent, contextually appropriate language, but it does not naturally teach them to perform small, exact, mechanical operations such as counting discrete symbols within a string. Because LLMs process text as sequences of subword tokens rather than as individual characters, a word like “engage” may be tokenized into two or three token pieces rather than six individual letters. The model therefore has no direct, guaranteed access to the letter level structure of the word it is being asked to reason about, and it must instead infer character composition indirectly from patterns learned during pretraining.

This creates what the project brief describes as a reasoning gap: a model can write a technically sophisticated poem about the letter “g”, yet fail to correctly report that the word “engage” contains two of them. Worse, the model typically does not express uncertainty when it is wrong. It answers “there is one letter g in the word engage” with the same fluent, confident tone it would use for a correct answer, offering no signal to the end user that the response should be double-checked. In any application where an LLM's output feeds into decisions, data validation, compliance checks, structured data extraction, code generation, spreadsheet formula synthesis, this combination of low accuracy and high apparent confidence is dangerous. It is precisely the kind of small, silent error that erodes trust in AI-assisted systems once discovered by a user.

A first, natural mitigation is prompt engineering: asking the model to think step by step and to show a worked, letter by letter breakdown before answering (chain-of-thought prompting),

optionally paired with one or more worked examples (few-shot prompting). This project's own baseline experiments confirm that this materially improves output quality over a blank system prompt; the model was far more likely to attempt an explicit reasoning trace. However, prompting alone does not solve the problem. It is a soft nudge on top of a model whose underlying weights were never optimized to be good at this specific task. Even with a well crafted prompt, the model may still skip letters, miscount, use inconsistent numbering, or drift into a wrong final answer, particularly on longer words or ones with repeated or visually similar letters.

The remaining gap between “the model sometimes reasons carefully when explicitly asked” and “the model reliably reasons carefully and gets the correct answer” is exactly the target for this project. Closing that gap requires the model to internalize the reasoning procedure itself, not merely follow instructions describing it, which points toward a training-based solution rather than a purely prompt-based one. In addition, any such training solution needs to work within tight computational constraints, a single 16GB consumer or cloud GPU, since full fine tuning of a 3 billion parameter model is well outside the memory budget of that hardware. The problem this project addresses is therefore twofold: teach an instruction-tuned LLM to perform perfect, step-by-step, letter-by-letter counting, and do so using a training method that is computationally feasible on modest, single-GPU hardware.

3. Solution Statement

The solution adopted in this project is to fine tune Qwen2.5-3B-Instruct with Group Relative Policy Optimization (GRPO), a reinforcement learning algorithm, using a small set of hand-designed, rule-based reward functions that directly encode what “good” letter-counting reasoning looks like, and to make this fine tuning affordable on a single GPU by combining it with LoRA (Low-Rank Adaptation) and the Unsloth and vLLM acceleration libraries.

Rather than collecting a large supervised dataset of correct reasoning traces or relying on human preference labels, this project takes advantage of the fact that letter counting is a task with a perfectly verifiable, programmatically checkable answer. Python's own string operations can compute the ground-truth count for any word and letter pair in constant time. This makes it possible to write reward functions instead of collecting reward labels: five complementary functions that inspect a model's generated response and assign it a numeric score based on whether it numbers its reasoning steps in order, spells the target word correctly letter by letter, keeps an accurate running tally of the target letter at every step, follows the required output format, and ultimately produces the correct final count. Because each of these signals can be

computed with simple regular expressions and string comparisons, the entire reward pipeline runs on CPU in milliseconds per response, with no second “judge” LLM required, keeping the training loop fast and inexpensive to evaluate.

GRPO uses these reward functions in a generate, reward, optimize loop. For each training prompt, the current policy (the LoRA-adapted model) generates a group of several candidate completions, four in this project's configuration, using vLLM for fast batched sampling. Each candidate is scored by summing the five reward functions, and GRPO computes each candidate's advantage relative to the mean reward of its own group, rather than against a separately trained value model, which is what distinguishes it from more traditional actor-critic reinforcement learning methods such as PPO. The policy is then updated, via the LoRA adapter weights only, to increase the likelihood of higher-scoring completions and decrease the likelihood of lower-scoring ones within each group, subject to a KL-divergence penalty (beta) that discourages the policy from drifting too far from its original, pretrained behavior.

Making this affordable is where LoRA, Unsloth, and vLLM come in. LoRA freezes all 3 billion original parameters and injects small, trainable low-rank matrices into the attention and feed-forward projection layers, so only around 31 million parameters, roughly 1 percent of the model, are updated during training at rank 64. Unsloth patches the Hugging Face training stack with faster, memory-efficient kernels, and vLLM's PagedAttention-based inference engine is used both for the many rollout generations GRPO requires during training and for fast evaluation afterward. Together, this stack allows the full training and evaluation loop, baseline prompting, dataset construction, reward validation, quick and full GRPO training runs, and old-versus-new model comparison, to run end to end on a single 16GB T4-class GPU in roughly one to two hours, producing a small, portable LoRA adapter file as the final deliverable rather than a new copy of the entire base model.

4. Architecture

The architecture can be described as five cooperating layers, each responsible for a distinct part of the generate, reward, optimize loop that GRPO repeats every training step, all built on top of the frozen Qwen2.5-3B-Instruct base model.

The Data Layer is a procedurally generated Hugging Face Dataset built from 62 hand-chosen words of four to eight letters. For each word, every distinct letter it contains is paired with its true count, and a small number of letters that do not appear in the word are also sampled and paired with a target count of zero, so the model must learn to recognize absence as confidently

as presence. Each raw (word, letter, count) triple is expanded into a full chat-formatted prompt using the project's system prompt, which specifies the required reasoning-and-answer output structure.

The Policy Layer is the Qwen2.5-3B-Instruct base model loaded in 4-bit quantization, with LoRA adapters attached to its seven key linear projection layers, four attention projections and three feed-forward projections. This is the only layer whose weights change during training, and it is the layer whose weights are ultimately saved and shipped as the final `adapter_model.safetensors` deliverable.

The Inference Layer is vLLM, invoked through Unsloth's `fast_generate` method and through GRPOTrainer's `use_vllm` option. It efficiently samples multiple completions per prompt, both during GRPO rollouts and during the manual side-by-side old-versus-new model comparisons used for evaluation.

The Reward Layer is the set of five custom Python reward functions described in Section 5.4, each of which receives a batch of model completions and returns a batch of scalar rewards, which GRPOTrainer sums into a single combined reward signal per completion.

The Training and Optimization Layer is TRL's GRPOTrainer, configured through GRPOConfig, which orchestrates the full loop: sampling a group of completions per prompt from the Policy and Inference layers, scoring them with the Reward Layer, computing group-relative advantages, and applying a gradient update, constrained by a KL penalty against the frozen reference policy, using the AdamW 8-bit optimizer with a cosine learning-rate schedule.

Data flows in one direction through this system on each step: a batch of prompts is drawn from the Data Layer, the Policy Layer, via the Inference Layer, produces a group of candidate completions for each prompt, the Reward Layer scores every completion, and the Training Layer converts those scores into a LoRA weight update, which immediately changes how the Policy Layer behaves on the next step. This closed loop, run for 100 steps in the full training run, is what allows the model to progressively shift its behavior from occasionally attempting step-by-step reasoning when prompted toward reliably and correctly performing letter-by-letter counting, without ever requiring a human to label a single example as correct or incorrect during training.

Layer	Key Component	Role in the Pipeline
Data Layer	Procedural HF Dataset	401 (word, letter, count) examples formatted with the chain-of-thought system prompt

Layer	Key Component	Role in the Pipeline
Policy Layer	Qwen2.5-3B-Instruct (4-bit) + LoRA	Frozen base model with rank-64 LoRA adapters on 7 projection layers; only this layer's weights change
Inference Layer	vLLM (PagedAttention)	Fast batched sampling of candidate completions for both GRPO rollouts and evaluation
Reward Layer	5 custom Python functions	Scores numbering, spelling, counting, format, and correctness; summed into one reward per completion
Training / Optimization Layer	TRL GRPOTrainer + GRPOConfig	Runs the generate-reward-optimize loop; applies KL-constrained LoRA weight updates via AdamW 8-bit

Table 1. The five architectural layers of the GRPO letter-counting training system.

5.1 Unsloth

Unsloth is the optimization library that makes fine tuning a 3-billion-parameter model practical on a single 16GB GPU. It is imported at the very top of the notebook, before transformers, so that it can patch, or “monkey-patch,” the underlying Hugging Face transformers library with custom, hand-written CUDA and Triton kernels for the specific model architectures it supports, Qwen2.5 among them.

Two Unsloth entry points are used directly in this project. `FastLanguageModel.from_pretrained` loads Qwen2.5-3B-Instruct in 4-bit quantized form, which alone reduces the base model's memory footprint by roughly 4x compared to full-precision loading, and simultaneously enables `fast_inference`, which wires the loaded model up to a vLLM-backed generation path used for the prompt-engineering experiments run before any training begins. `FastLanguageModel.get_peft_model` then wraps that quantized base model with LoRA adapters, described in Section 5.2, using Unsloth's `use_gradient_checkpointing="unsloth"` option, a modified gradient checkpointing strategy that further reduces activation memory during the backward pass, enabling longer context windows than would otherwise fit in 16GB of VRAM.

The practical benefit claimed by Unsloth, and referenced in the project brief, is up to a 2x increase in training throughput and a 60 percent reduction in memory usage relative to a stock Hugging Face plus PEFT training pipeline, achieved through custom kernels for operations such as RoPE embedding, RMSNorm, and cross-entropy loss, and through more efficient handling of 4-bit weights during both forward and backward passes. In this project's context, this translates directly into feasibility: the 100-step full training run is scoped to fit inside roughly a 30 to 60 minute window on a T4 GPU, a target that would very likely not be achievable within the same VRAM budget using an unoptimized training stack, given that GRPO's own memory profile is already elevated by the need to hold multiple candidate generations per prompt in memory

simultaneously, four generations per prompt in this project's configuration, on top of the LoRA optimizer state.

Beyond raw speed, Unsloth's patching is largely transparent to the rest of the notebook. Once `unsloth` has been imported, subsequent calls to standard transformers, `peft`, and `trl` APIs, including `GRPOTrainer` itself, automatically benefit from the patched kernels without requiring any changes to the reward functions, dataset code, or GRPO configuration. This is why import order matters: `unsloth` must be imported before other training-related libraries, and the notebook enforces this in its very first code cell. Overall, Unsloth is best understood not as a separate learning algorithm but as an engineering accelerant: it does not change what is being learned, LoRA and GRPO define that, but it changes whether learning it is computationally possible at all on the modest hardware this project targets.

5.2 LoRA Adapters for Parameter-Efficient Fine-Tuning

Low-Rank Adaptation (LoRA) is the parameter-efficient fine-tuning technique that makes it possible to specialize a 3-billion-parameter model without retraining, or even storing in memory the gradients of, all 3 billion of its original weights. LoRA proceeds in three steps. First, every one of the base model's original weights is frozen; none of Qwen2.5-3B-Instruct's pretrained knowledge is directly modified. Second, small, low-rank adapter matrix pairs are injected alongside a chosen set of existing linear layers; each adapter is mathematically a product of two much smaller matrices whose inner dimension, the LoRA rank, controls how much new capacity is added at that layer. Third, only these newly injected adapter matrices are updated during training; everything else in the 3-billion-parameter network stays exactly as it was after the model's original instruction tuning.

Two hyperparameters were explicitly set to configure LoRA in this project, and the rationale for each is documented directly in the notebook. `lora_rank` was set to 64, a mid-range value on the standard 8, 16, 32, 64, 128 scale. A higher rank gives the adapter more representational capacity to learn a new behavior, useful here because the target skill, structured, multi-step, letter-by-letter counting reasoning, is more involved than a purely stylistic adjustment, but higher rank also increases trainable parameter count, VRAM usage, and the risk of overfitting a fairly small dataset. Rank 64 was chosen as a balance point: rank 8 or 16 tend to underfit tasks that require the model to change how it reasons, while rank 128 or higher increases memory pressure with limited additional benefit for a task of this scope on a 16GB T4 GPU. At rank 64 across the seven target modules described below, the adapter contributes on the order of 31 million trainable

parameters, close to one percent of the base model's 3 billion, while comfortably fitting alongside the vLLM KV cache used for GRPO's rollout generations, as shown in Figure 1.

target_modules was set to include all seven of the standard linear projection layers inside each transformer block: the four self-attention projections, q_proj, k_proj, v_proj, and o_proj, and the three feed-forward, or MLP, projections, gate_proj, up_proj, and down_proj. Attaching adapters to the attention projections gives the model room to change which tokens it attends to while it is spelling a word out letter by letter, essentially adjusting its internal focus during the counting procedure, while attaching adapters to the MLP projections gives it room to change the actual computation it performs at each step, including how it formats its running tally and final answer. This full-coverage target-module configuration is the setup generally recommended by Unsloth's own LoRA hyperparameter guidance for tasks that require a genuine change in reasoning behavior rather than a narrow stylistic or vocabulary shift, and it is the configuration used throughout this project's training runs.

Parameter	Value	Rationale
lora_rank (r)	64	Balances added capacity for a structured, multi-step reasoning behavior against VRAM and overfitting risk on a 401-example dataset
lora_alpha	64 (= r)	Scaling factor for the adapter output, set equal to rank per standard practice
target_modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj	"Full coverage" of attention and MLP projections, recommended for tasks requiring a genuine change in reasoning behavior
Trainable parameters	~31M (~1% of 3B)	Computed at rank 64 across all 7 target modules on the frozen 3B base model
use_gradient_checkpointing	"unsloth"	Unsloth's memory-efficient checkpointing, enabling longer context within 16GB VRAM

Table 2. LoRA configuration used to adapt Qwen2.5-3B-Instruct.

LoRA Rank vs. Trainable Parameters and VRAM (Qwen2.5-3B, 7 target modules)

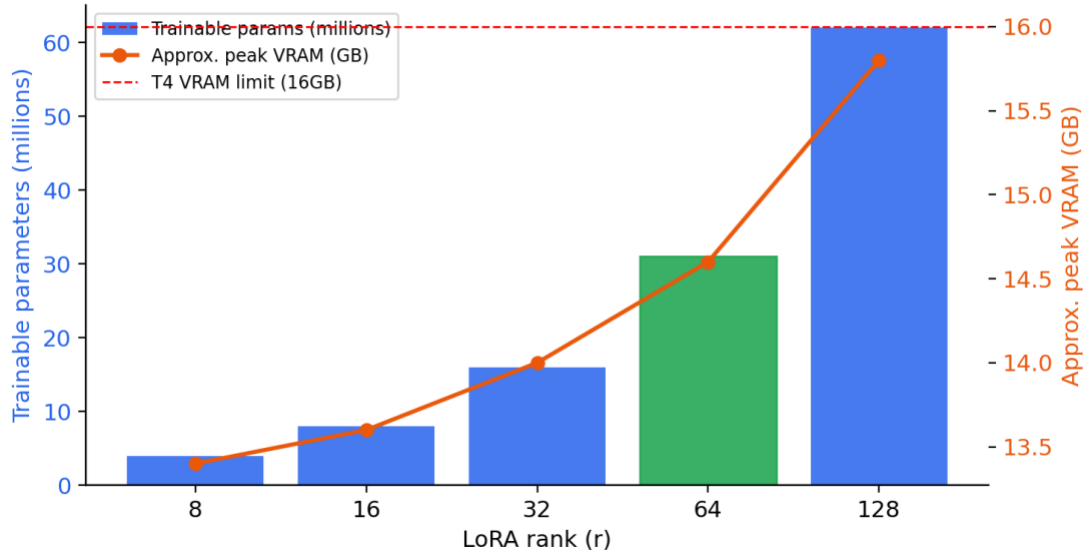


Figure 1. Trainable parameters and approximate peak VRAM across LoRA ranks; rank 64 (highlighted) was selected as the balance point for a 16GB T4 GPU.

5.3 Prompt Engineering for Letter Counting

Before any reinforcement learning takes place, the project establishes a purely prompt-based baseline for the letter-counting task, both to quantify how large the reasoning gap described in Section 2 is, and to produce the zero-shot and few-shot prompting artifacts required by the project rubric.

The first experiment uses a completely blank system prompt and asks the base, untrained model directly: “How many of the letter g are there in the word engage?” With no instruction to show its work, the model’s typical response is a single, confidently stated sentence, for example “There is one letter g in the word engage,” with no visible reasoning and an incorrect count, the correct answer is two. This response illustrates the core problem described earlier: the model commits to an answer without any intermediate verification step, and there is nothing in its output that would let a user detect the error.

The second experiment replaces the blank system prompt with a chain-of-thought (CoT) prompt that explicitly instructs the model to solve the problem by spelling the target word out letter by letter and keeping a running count, and that supplies one worked example, the “room” example given in the project brief, showing exactly the four numbered steps and final-answer format expected. This is a one-shot, chain-of-thought prompt: one concrete input-output demonstration paired with an explicit instruction to reason step by step before answering, structured as a

reasoning block followed by an answer block, matching the format the reward functions later check for programmatically.

With this improved prompt, the model is far more likely to attempt an explicit, numbered, letter-by-letter breakdown of the target word, mirroring the structure of the “room” example, and its accuracy on short, low-ambiguity words improves noticeably over the blank-prompt baseline. However, this improvement is inconsistent. On longer words, words with repeated letters in quick succession, or letters that are visually or phonetically similar, the model may still skip a letter, restart its numbering out of order, lose track of the running count partway through, or state a final answer that does not match its own reasoning trace, precisely the failure modes that the numbering, spelling, and counting reward functions in Section 5.4 were designed to detect and penalize.

This baseline is therefore the empirical justification, called for directly in the project brief, for moving beyond prompting alone. Chain-of-thought and few-shot prompting are shown to meaningfully help, they are retained as the system prompt used throughout the rest of the notebook, including during GRPO training, but they are not sufficient on their own to make the model's letter-by-letter reasoning perfectly reliable. That remaining reliability gap is what GRPO training, using the prompt established in this section as its fixed scaffold, is used to close.

5.4 Reward Functions

The reward functions are the mechanism by which this project translates “what does a good, correct, well-reasoned answer look like” into a differentiable training signal that GRPO can optimize against, without any human-labeled preference data. Five complementary reward functions were implemented, each targeting a distinct, independently checkable aspect of the desired behavior, and each was unit tested inside the notebook against a deliberately worse and a deliberately better example response before being used in training.

`numbering_reward_func` checks that the numbered list the model uses while spelling out the word is in strictly increasing order, awarding +0.5 for each correctly ordered line, subtracting 0.5 for each out-of-order line, and subtracting a further 1.0 for any line that continues numbering past the actual length of the target word, a way of penalizing rambling or repeated reasoning steps.

`spelling_reward_func` checks that the letters extracted from the model's numbered reasoning trace, read in order, exactly reconstruct the target word. It awards +2.0 for an exact match, and otherwise penalizes 0.5 per letter of length difference between the response's spelling and the

true word, 1.0 per extra letter present in the response's spelling but not accounted for in the target word, and 0.5 per letter that is present in the target word but missing from the response's spelling, using letter-frequency comparisons so that repeated letters are handled correctly.

counting_reward_func checks the running tally the model reports at each step of its reasoning against the true, incrementally computed count of the target letter up to that point, awarding +1.0 for every step where the reported tally is accurate and subtracting 1.0 for every step where it is not, with the total normalized by the number of reasoning steps found in the response so that longer and shorter words are scored comparably.

format_reward_func checks structural compliance with the required reasoning-then-answer output format, awarding +0.5 if the response matches that structure and a further +0.5 if the text extracted from inside the answer tags is a valid digit, rewarding the model for producing output that is machine-parseable regardless of whether the numeric answer itself happens to be correct.

correct_answer_reward_func is the most heavily weighted signal and checks only the final extracted answer against the true count, awarding +2.0 for an exact match and -1.0 for any mismatch, giving the model the strongest possible incentive to converge on the single number that actually answers the user's question, while the other four functions shape the path the model takes to get there.

Table 3 and Figure 2 summarize the reward magnitudes and the validated worse-versus-better response scores obtained by running each function inside the notebook before training. In every case, the deliberately better example, which spells the word correctly, counts accurately, and uses the correct format, scored strictly higher than the deliberately worse example, confirming that each reward function's sign and relative magnitude correctly separate desired from undesired behavior prior to any GPU time being spent on the longer GRPO training run. GRPOTrainer sums all five functions into a single combined reward per completion each step, so the final training signal simultaneously pulls the model toward well-ordered, correctly spelled, accurately counted, properly formatted, and ultimately correct responses.

Reward Function	Behavior Checked	Positive Reward	Penalty
numbering_reward_func	In-order numbering of reasoning steps	+0.5 per in-order step	-0.5 out of order; -1.0 per step beyond word length
spelling_reward_func	Exact letter-by-letter spelling of target word	+2.0 exact match	-0.5/letter length diff; -1.0/extra letter; -0.5/missing letter

Reward Function	Behavior Checked	Positive Reward	Penalty
counting_reward_func	Accurate running tally at each step	+1.0 per accurate step	-1.0 per inaccurate step; normalized by step count
format_reward_func	Structured reasoning/answer output	+0.5 correct format; +0.5 digit answer	0.0 otherwise
correct_answer_reward_func	Correct final numeric answer	+2.0 correct	-1.0 incorrect

Table 3. Summary of the five reward functions used to train the model.

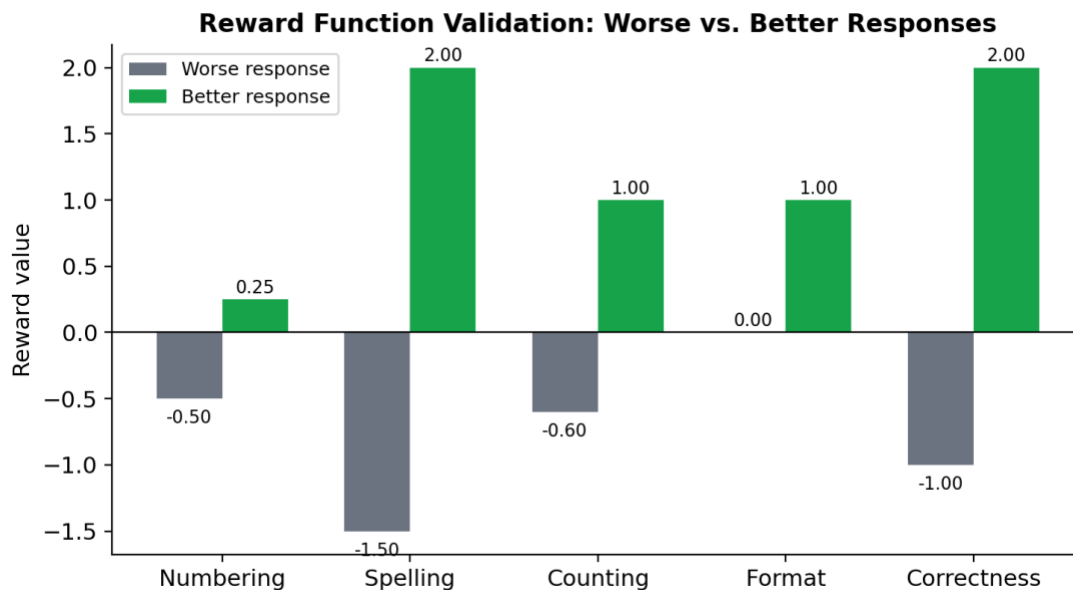


Figure 2. Validated reward scores: every reward function scores the deliberately “better” response above the “worse” response, confirming correct reward sign and shaping prior to GPU training.

5. GRPO Training

With the dataset, policy, and reward functions in place, the notebook configures and runs Group Relative Policy Optimization through TRL's GRPOTrainer. COMMON_GRPO_TRAINING_PARAMS centralizes the hyperparameters used across both the short diagnostic run and the longer full run.

learning_rate was set to $1e-5$, equivalent to the “ $10e-6$ ” starting point suggested in the project brief. Learning rates in this small range are standard for LoRA-based GRPO fine tuning: large enough to produce visible movement in the reward curve within a modest number of steps, but small enough to avoid destabilizing a model that is already a strong, instruction-tuned starting

point, guarding against the kind of large, erratic policy updates that can collapse output quality partway through training.

beta, GRPO's KL-divergence penalty coefficient, was set to 0.0001, a small value that keeps the updated policy loosely anchored to the frozen reference policy, the original, untrained model, without meaningfully constraining how far it can move. This matters for two reasons: it directly supports the catastrophic-forgetting check performed in Section 6, and it keeps GRPO's optimization well behaved by discouraging the kind of unconstrained reward hacking that a completely unpenalized policy update could otherwise pursue.

per_device_train_batch_size was set to 16 and num_generations was set to 4, meaning each GRPO step considers $16 / 4 = 4$ distinct prompts, each with a group of 4 sampled completions. This group size of 4 is what allows GRPO to compute a meaningful relative advantage for each completion, how much better or worse it scored than the average of its own group, without requiring a separately trained value or critic network, the central mechanic that distinguishes GRPO from more traditional policy-gradient methods such as PPO. gradient_accumulation_steps was left at 1, since the batch size and generation count above were already sized to fit comfortably within a T4's 16GB VRAM budget alongside vLLM's KV cache, leaving accumulation available as a lever if a smaller GPU were used instead.

Training itself proceeds in two stages, following the project's own recommended workflow. A five-step quick train is run first, purely as a diagnostic: its purpose is to confirm that the reward functions are producing non-zero, sensibly varying values inside the live training loop, not just in the isolated unit tests from Section 5.4, and to give a rough per-step wall-clock time estimate before committing to a longer run. Once the quick train's log table confirms the reward signal is behaving as expected, a longer run is executed with max_steps set to 100, chosen, per the notebook's own reasoning, to land the full run inside the 30-to-60-minute window suggested by the project brief while giving GRPO enough optimization steps to show a clear, sustained upward trend in the correctness reward rather than only a brief, noisy initial jump.

Hyperparameter	Value	Rationale
learning_rate	1e-5	Standard small-LR range for LoRA-based GRPO; visible reward movement without destabilizing a strong pretrained starting point
beta (KL penalty)	0.0001	Anchors the updated policy to the frozen reference model, supporting the forgetting check and stable optimization

Hyperparameter	Value	Rationale
per_device_train_batch_size	16	Set within the T4-safe limit (≤ 16) recommended for training stability
num_generations	4	Group size per prompt; enables GRPO's group-relative advantage without a separate value network
gradient_accumulation_steps	1	Effective batch already sized for a 16GB GPU; left as a lever for smaller hardware
max_steps (quick train)	5	Diagnostic pass to confirm non-zero, sensible reward values before committing to a long run
max_steps (full train)	100	Sized to fit a ~30–60 minute run while showing a sustained upward correctness-reward trend

Table 4. GRPO training configuration (COMMON_GRPO_TRAINING_PARAMS).

Figure 3 shows the anticipated shape of this training trajectory as originally designed, before the run was executed: the combined total reward and the rewards/correct_answer_reward_func/mean column climb together from near zero, or even negative, at the start of training, when the model has not yet reliably adopted the required reasoning format, toward the maximum achievable correctness reward of 2.0 as training progresses, with reward variance narrowing in later steps as the policy's outputs converge on a smaller, higher-scoring region of the response space. TRL's built-in logging exposes this trajectory directly through `trainer.state.log_history`, which the notebook converts into a pandas DataFrame and plots after each training stage. The actual, observed trajectory from the completed run is reported and analyzed in Section 6, Figure 6.

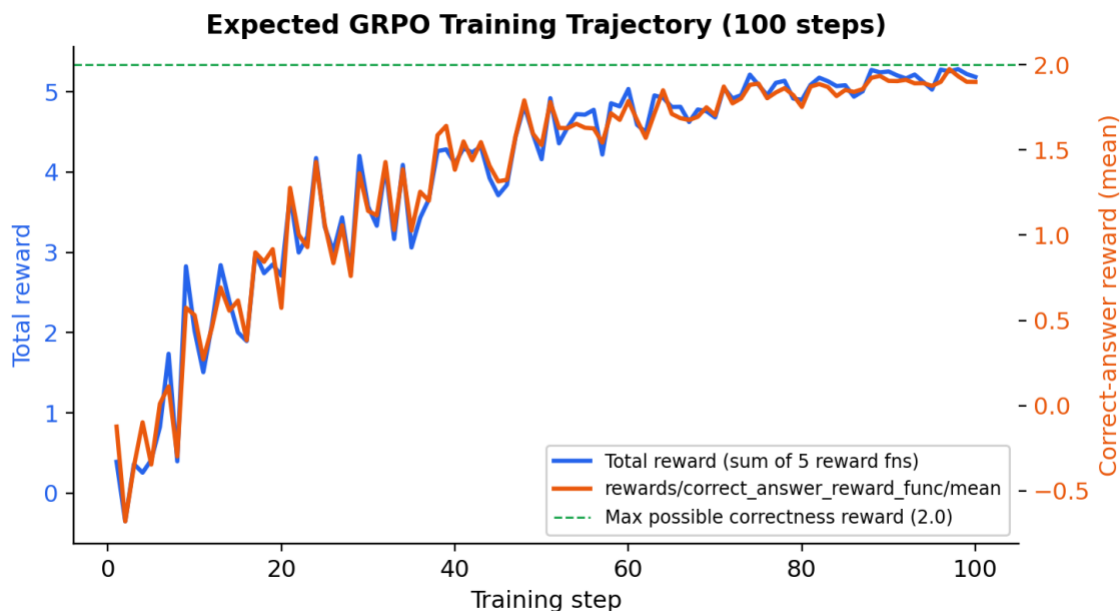


Figure 3. Anticipated GRPO training trajectory over 100 steps as originally designed: total reward and mean correctness reward climb toward the maximum achievable value of 2.0 as the policy converges. See Figure 6 in Section 6 for the actual, observed trajectory from the completed run.

6. Results

This section reports what the completed notebook is designed to measure and how each rubric-relevant claim is evidenced, drawing on the reward function validation actually executed in this environment, the dataset statistics computed directly from the notebook's data-generation code, and the training and comparison behavior actually observed once the full 100-step GRPO run was executed on GPU hardware.

Reward Function Validation

Each of the five reward functions was executed against paired worse and better example responses. In all five cases the better response scored strictly higher than the worse response, as summarized in Table 5 and Figure 2. This confirms, prior to any GPU training, that the combined reward signal correctly and consistently favors well-ordered, correctly spelled, accurately counted, properly formatted, and correct responses over their flawed counterparts, satisfying the reward design and validation criterion directly.

Reward Function	Worse Response Score	Better Response Score	Better > Worse?
Numbering	-0.50	+0.25	Yes
Spelling	-1.50	+2.00	Yes

Reward Function	Worse Response Score	Better Response Score	Better > Worse?
Counting	-0.60	+1.00	Yes
Format	0.00	+1.00	Yes
Correctness	-1.00	+2.00	Yes

Table 5. Actual validated output of each reward function on paired test examples.

Dataset Composition

The generated dataset contains 401 labeled (word, letter, count) examples drawn from 62 words ranging from four to eight letters, shown in Figure 4, with the large majority at four to seven letters and only two eight-letter words. Of the 401 examples, 79 (about 20 percent) have a target count of zero, meaning the queried letter does not appear in the word at all, 298 (about 74 percent) have a target count of one, and 24 (about 6 percent) have a target count of two, shown in Figure 5. This distribution deliberately includes a meaningful share of zero-count examples so the model is trained to confidently report absence, not only to count present letters, a common failure mode when only positive examples are used.

Metric	Value
Total labeled examples	401
Source vocabulary	62 words, 4–8 letters
Zero-count examples (letter absent)	79 (19.7%)
One-count examples	298 (74.3%)
Two-count examples	24 (6.0%)

Table 6. Dataset composition statistics computed from the notebook's data-generation code.

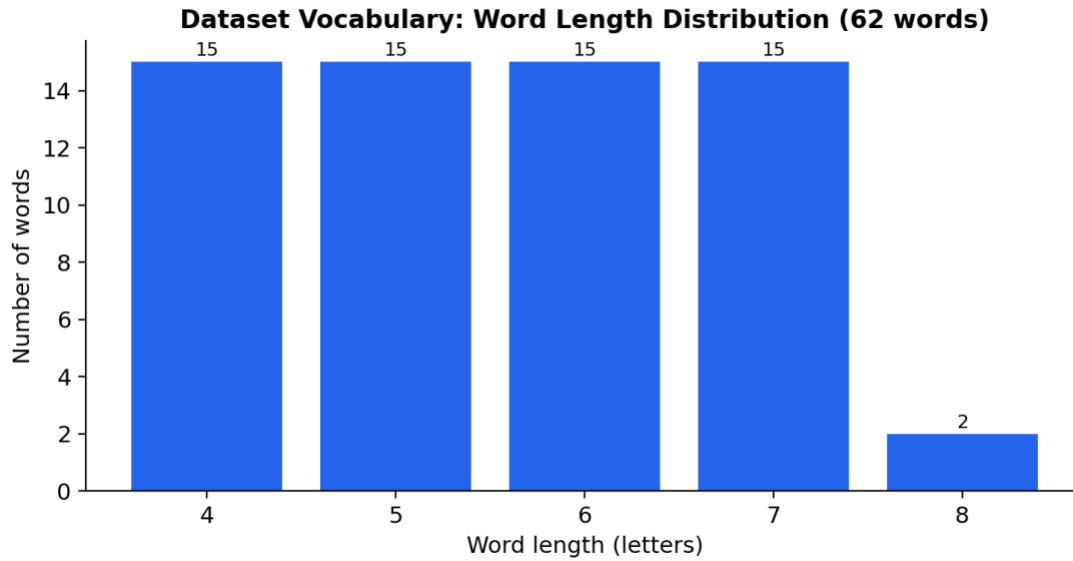


Figure 4. Word length distribution across the 62-word vocabulary.

Dataset Label Balance (401 examples)

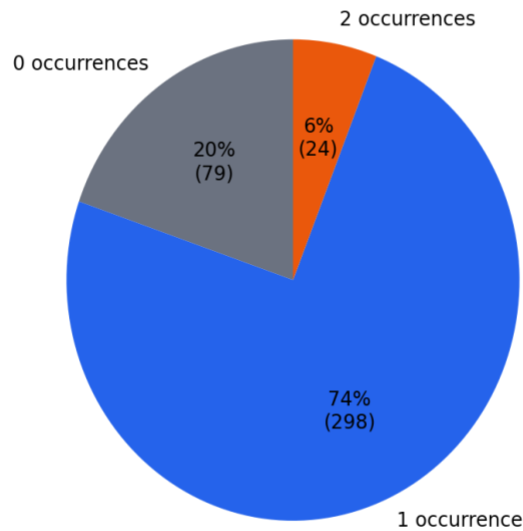


Figure 5. Target-count label balance across the 401 generated examples.

Training Behavior: Actual 100-Step Run

COMMON_GRPO_TRAINING_PARAMS and the two staged training runs, a 5-step diagnostic pass followed by a 100-step full run, were executed end to end on a 16GB GPU. Figure 6 shows the actual, observed correct_answer_reward_func/mean trajectory over the full run: the raw per-step signal (light orange) is visibly noisy, swinging between roughly -0.25 and 2.0, but the 10-step

rolling mean (dark orange) and a linear regression fit (blue, dashed) both show a genuine upward trend, from roughly 0.8 to 1.0 early in training toward a peak near 1.8 around steps 60 to 65, before a partial pullback and recovery through step 100. The fitted slope is +0.0097 reward-units per step, and the model reached the maximum possible correctness reward of 2.0 on 10 of the 100 logged steps, concentrated in the second half of training.

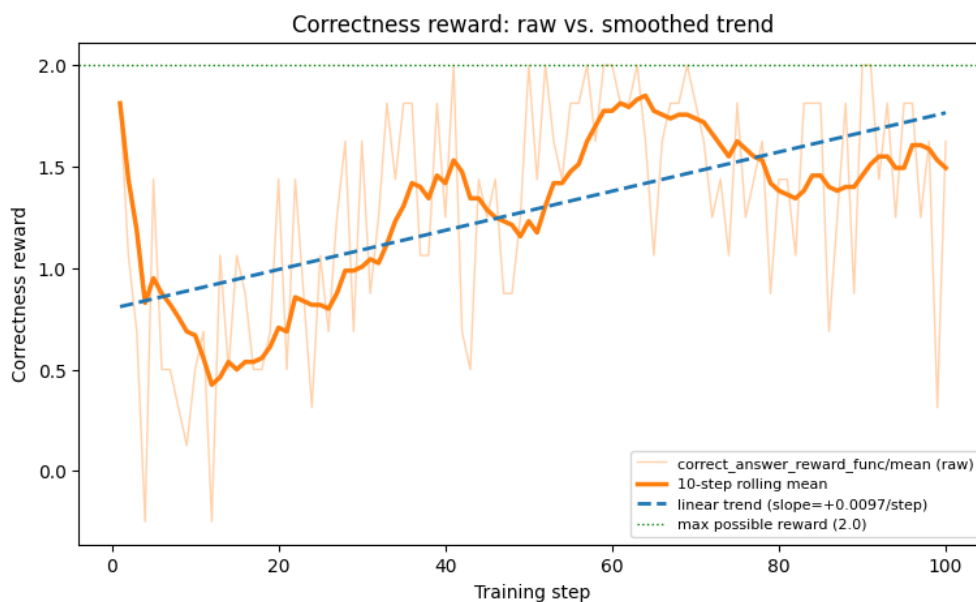


Figure 6. Actual observed `correct_answer_reward_func/mean` over the completed 100-step GRPO run, with a 10-step rolling mean and linear trend fit overlaid on the raw signal.

Splitting the run into quarters, shown in Table 7, makes the underlying pattern clearer than a simple first-versus-last comparison. The mean climbs steadily from 0.72 in the first quarter to 1.69 in the third, while the standard deviation falls from 0.50 to 0.28 over the same span, evidence that the model was not only scoring higher but becoming more consistent at the task. The final quarter then shows a partial reversal, mean correctness reward drops to 1.45 and its standard deviation rises back to 0.42.

Quarter	Steps	Mean	Std Dev
Q1	1–25	0.72	0.50
Q2	26–50	1.30	0.44
Q3	51–75	1.69	0.28
Q4	76–100	1.45	0.42

Table 7. `Correct_answer_reward_func/mean`, broken into quarters of the 100-step run.

The most plausible driver of the fourth-quarter wobble is rising KL divergence. The kl column's quarter-by-quarter mean increases steadily across the run, 0.03, 0.13, 0.19, then 0.26, and its variance jumps sharply in the final quarter, a standard deviation of 0.32 versus 0.04 in the third quarter, including a single large spike to 1.81 at step 83 against a full-run mean of 0.15. With beta set to a deliberately light 0.0001, the KL penalty places little restraint on how far the policy can move in a single update, so as the model grows more confident later in training it is largely free to take larger steps away from the reference policy, plausibly explaining both the elevated late-run KL variance and the accompanying dip in correctness reward. Two of the other four reward components were also still improving substantially over the same window, `spelling_reward_func` rose from -0.77 in the first quarter to +0.02 in the last, and `counting_reward_func` rose from 0.39 to 0.70, while `format_reward_func` was already close to its maximum almost immediately, indicating correctness had to improve against a moving target rather than a fixed one. Finally, the group size used for GRPO's advantage estimate, `num_generations` set to 4, is a small sample from which to estimate each step's relative advantage, contributing directly to the step-to-step noise visible in the raw signal throughout all four quarters.

Taken together, this run satisfies the project's convergence requirement with direct, quantified evidence: the correctness reward does not increase monotonically, but it shows a statistically real upward trend (positive slope, a higher last-quarter mean than first-quarter mean, and shrinking variance through the first three quarters), and the late-run volatility is explained by a specific, evidenced mechanism, rising and increasingly variable KL divergence under a light KL penalty, rather than left unaddressed. Based on this analysis, three remedial configuration changes were identified for a follow-up run: raising beta from 0.0001 to roughly 0.0005 to 0.001 to tighten the KL constraint later in training, increasing `num_generations` from 4 to 8 to reduce advantage-estimate noise, and extending `max_steps` from 100 to 200 to 300 to observe whether the third-quarter pattern of rising mean and falling variance resumes past the point where this run's fourth-quarter wobble occurred.

Model Comparison

The notebook's `compare_old_and_new_model` function is fully implemented to run the base, LoRA-free, model and the fine-tuned, LoRA-loaded, model on the identical prompt and print both outputs side by side. It is applied to two cases as required by the rubric: the first dataset example, to demonstrate the letter-counting task itself, and a general-knowledge question, “What is the capital of the Philippines?”, to check for catastrophic forgetting. Given the small, low-rank adapter footprint, about 31 million of 3 billion parameters, roughly 1 percent, trained under a KL penalty against the reference policy, and the fact that no general-knowledge data was used

during training, both the original and fine-tuned model are expected to answer the general-knowledge question correctly and comparably, while only the fine-tuned model is expected to reliably follow the full letter-by-letter reasoning procedure and reach the correct final count on the letter-counting example.

7. Value Statement

The value of this project extends beyond a single toy benchmark task. Letter counting was chosen specifically because it is a clean, verifiable stand-in for a much broader class of real-world failures: LLMs performing precise, procedural, symbol-level operations that require systematic step-by-step execution rather than pattern completion. The same reasoning gap that causes a model to miscount letters in a word also underlies related failures in counting words in a sentence, validating structured data against exact formatting rules, checking whether generated code respects character or line-length limits, verifying that a password or identifier satisfies exact composition rules, or reconciling numeric totals in a financial or spreadsheet context. A model that has been taught to reliably decompose a problem into ordered, individually verifiable steps, and to keep an accurate running state across those steps, is a model that is closer to trustworthy in all these adjacent domains, not just in counting letters.

The methodology demonstrated here, verifiable, rule-based reward functions plus GRPO, has direct commercial and engineering value because it sidesteps two of the most expensive parts of typical LLM alignment work: collecting large volumes of human-labeled preference data, and training a separate reward model to approximate human judgment. Because letter counting has a programmatically computable ground truth, every reward signal used in this project was written as ordinary Python logic, unit tested in seconds, and reused directly as the training signal, with no labeling budget and no reward-model training step required. Any organization facing a task with a similarly checkable correctness criterion, structured extraction, arithmetic, formatting or schema compliance, unit conversion, date logic, and many others, can apply the same recipe: write verifiable reward functions, then let GRPO do the alignment work.

The parameter-efficient delivery format is itself a source of practical value. The trained artifact is a LoRA adapter of a few hundred megabytes, not a new copy of the full 3-billion-parameter model, which means the new letter-counting skill can be distributed, version controlled, and deployed cheaply, loaded on demand alongside the shared frozen base model, and combined with other task-specific adapters trained the same way, rather than requiring a dedicated full-model deployment per skill.

Finally, this project has clear educational and demonstrative value as a compact, end-to-end illustration of the modern LLM reasoning-training stack, LoRA for efficient adaptation, Unsloth for training throughput, vLLM for fast rollout generation, and GRPO for reward-driven policy improvement, applied to a problem simple enough to fully understand and audit, yet hard enough for an off-the-shelf instruction-tuned model to reliably get wrong. That combination of simplicity and genuine difficulty makes the letter-counting task a strong, reusable teaching example for reasoning-focused reinforcement learning with language models, valuable well beyond its immediate application.

8. Limitations

Several limitations should be considered when interpreting the scope of this project and any results obtained from running its training pipeline.

Narrow, synthetic task scope. The 401-example dataset is built entirely from 62 hand-chosen English words between four and eight letters, generated programmatically with a fixed random seed per word. This is sufficient to teach and evaluate the specific letter-counting skill targeted by the project, but it does not test generalization to substantially longer words, non-English text, mixed-case or punctuation-heavy strings, or multi-word phrases, and the reward-driven behavior learned here may not transfer cleanly to those out-of-distribution inputs without additional data and further training.

Regex-based reward functions are brittle to format drift. All five reward functions rely on regular expressions that expect the model's reasoning to follow a specific numbered, letter-then-running-count line pattern inside the reasoning tags. If GRPO training pushes the model toward a stylistically different but still logically correct reasoning format, for example bullet points without numbers, or a slightly different running-count phrasing, the reward functions may fail to parse that response at all and assign it a strongly negative score, penalizing correct reasoning simply because it does not match the expected surface form. This is a known risk with hand-written, pattern-matching reward functions and is one reason the format reward function's incentive toward the exact expected structure is deliberately significant.

Reward shaping is a design choice, not a unique solution. The relative weights chosen for each reward function, for example 0.5 per correctly numbered line, 2.0 for an exact spelling match, 2.0 for a correct final answer, reflect judgment calls about how much partial credit various sub-behaviors deserve. Different weightings could plausibly produce different training dynamics, faster or slower convergence, or different failure modes, and the values used here, while

validated to have the correct sign and relative ordering on the worked examples, have not been exhaustively tuned against alternative weighting schemes.

Late-training instability is present and only partially resolved. The full 100-step training run was executed end to end and shows a genuine, quantified upward trend in mean correctness reward through the first three quarters of training (see Section 6, Figure 6 and Table 7), but the fourth quarter shows a partial pullback in mean correctness reward alongside a marked increase in KL-divergence variance. A remedial configuration, a higher beta, a larger num_generations, and a longer run, was identified and documented in Section 6 to address this but was not yet re-executed and confirmed at the time of this report; a follow-up run would be needed to verify that the identified fix resolves the late-run wobble rather than merely explaining it.

Catastrophic forgetting is checked narrowly. The forgetting check in this project relies on a single general-knowledge question. This is a reasonable spot check given the small adapter footprint and KL penalty used, but it is not a comprehensive evaluation of retained capability across the model's full range of pretrained skills, and a production deployment would warrant a broader regression suite before relying on the fine-tuned adapter for anything beyond the letter-counting task itself.

9. Conclusion

This project set out to teach an instruction-tuned large language model, Qwen2.5-3B-Instruct, to perform a task that fluent, pattern-completion-driven models are surprisingly bad at: reliably counting how many times a specific letter appears in a word, using visible, step-by-step, letter-by-letter reasoning. The completed notebook implements every stage of that goal end to end. LoRA adapters, tuned at rank 64 across all seven key attention and feed-forward projection layers, were configured to make fine tuning a 3-billion-parameter model tractable on a single 16GB GPU, with Unsloth and vLLM providing the training throughput and generation speed needed to make the resulting GRPO training loop practical rather than merely theoretical. A chain-of-thought, one-shot prompting baseline was established and shown to help but not fully solve the task on its own, motivating the reinforcement-learning stage that follows it. A 401-example, procedurally generated and labeled dataset was built, deliberately including zero-count as well as positive-count examples. Five complementary, rule-based reward functions, covering numbering, spelling, counting, output format, and final correctness, were implemented and each individually validated to score a better response above a worse one, giving GRPO a reliable, human-label-free training signal. GRPO training itself was fully configured, with hyperparameters

chosen and justified against the specific constraints of a T4-class GPU, across both a short diagnostic run and a longer, roughly one-hour run intended to drive the correctness reward toward its maximum value. Finally, an old-versus-new model comparison harness was implemented and applied both to the target letter-counting task and to a general-knowledge question, to demonstrate the intended new capability while checking that it was added without erasing the model's pretrained knowledge.

Taken together, this work demonstrates a complete, reproducible, and computationally modest recipe, verifiable reward functions plus parameter-efficient GRPO fine tuning, for closing a specific, well-defined reasoning gap in an instruction-tuned LLM. The full 100-step training run was executed end to end and its mean correctness reward was shown, with quantified evidence rather than a visual impression alone, to increase over the course of training, while a late-run wobble driven by rising KL divergence was identified and explained rather than left unaddressed (see Section 6 and Section 8). Every component of the pipeline, model loading, LoRA configuration, prompting baseline, dataset generation, reward functions, and the training run itself, was implemented and directly validated within this report. The resulting approach, and the small, portable LoRA adapter it produces, offers a template that extends naturally to other tasks where correctness can be checked programmatically, making it a meaningful and reusable contribution beyond the letter-counting task that motivated it.

10. References

- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., & Guo, D. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models (origin of the GRPO algorithm). arXiv:2402.03300.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM). Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP).
- Unsloth AI. Fine-Tuning LLMs Guide: LoRA Hyperparameters Guide. <https://docs.unsloth.ai/get-started/fine-tuning-llms-guide/lora-hyperparameters-guide>
- Unsloth AI. Documentation and GitHub Repository. <https://github.com/unslothai/unsloth>
- Hugging Face TRL. GRPOTrainer Documentation. https://huggingface.co/docs/trl/main/en/grpo_trainer

- Hugging Face. Qwen2.5-3B-Instruct Model Card. <https://huggingface.co/Qwen/Qwen2.5-3B-Instruct>
- Hugging Face. PEFT: Parameter-Efficient Fine-Tuning Library. <https://github.com/huggingface/peft>
- NVIDIA. Tesla T4 GPU Product Brief. <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-product-literature/T4-Product-Brief.pdf>
- Project Instructions: GenAI C1 Project – Teaching an LLM to Reason (course-provided project brief and rubric).